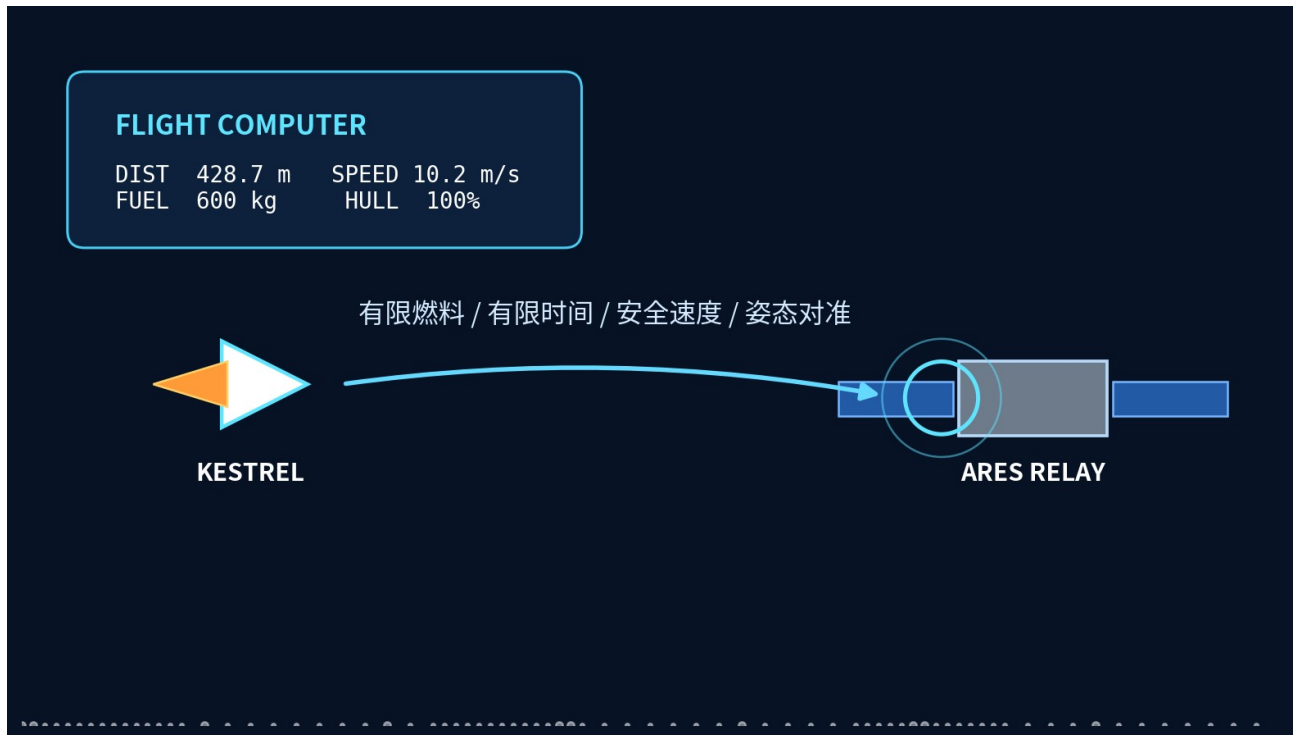


ORBITAL RESCUE

轨道救援 - C++ GUI 飞船对接模拟器

项目任务指导书



项目定位 这是一个“已有可运行代码 + 需要逐步接管”的综合工程。目标不是复制答案，而是测试你能否把 C++ 基础、GUI、物理模拟、状态机和调试组织成一个完整程序。

适用环境	Windows + CLion + CMake + C++20
GUI 库	raylib 5.5 (由 CMake FetchContent 下载)
建议周期	8-12 次开发，每次约 1.5-3 小时
版本	1.0 / 2026-08

目录与使用方式

部分	内容	主要产出
A	项目说明、基线运行、工程结构	确认能编译并理解程序流
B	阶段 1：配置、重构、轨迹	掌握文件 I/O、struct、vector
C	阶段 2：碰撞、引力、时间倍率	掌握向量物理与数值稳定
D	阶段 3：GUI 与多页面状态	实现真正可点击的应用界面
E	阶段 4：自动对接	FSM + PD 控制器
F	故障系统、测试、评分、答辩	形成可评估的完整工程

正确使用方法 先完成“验收条件”，再看下一任务。指导书给出接口、算法步骤和测试方法，但故意不提供可直接粘贴的完整答案。

建议每次开发都保留三个证据：

- 本次修改前后的 Git commit；
- 一张运行截图或一段 20-40 秒录屏；
- 一段简短记录：遇到什么问题、如何定位、最终如何解决。

1. 项目任务与成功标准

1.1 情景设定

微流星撞击使 ARES Relay 空间站失去姿态控制。救援飞船 Kestrel 必须独立完成最后数百米的交会与对接。空间站无法转向配合，飞船又受到燃料、生命保障时间、碰撞速度与对接姿态的共同限制。

你接手的不是空白工程。程序已经能够显示飞船、空间站、遥测数据，并允许键盘操纵。你的任务是把它从“可玩的原型”升级为结构清楚、物理可信、GUI 完整、能够自动对接和处理故障的工程。

1.2 最终必须实现的体验

- **手动飞行：**玩家可用推进器和姿态控制完成对接。
- **任务选择：**至少三种初始条件和难度。
- **GUI 操作：**开始、暂停、重开、自动驾驶均可鼠标点击。
- **物理效果：**碰撞会反弹并受损；可选月球引力；时间倍率可切换。
- **自动对接：**自动驾驶必须通过 ControlInput 操纵推进器，不能瞬移。
- **任务结果：**显示用时、耗油、最大速度、撞击速度和总分。
- **故障挑战：**至少实现一种会真实改变飞行行为的故障。

1.3 最终通过标准

- ☐ 从干净目录打开工程后能够正常配置、编译和运行。
- ☐ 不存在明显崩溃、数组越界、除零或无限循环。
- ☐ 手动和自动方式均至少能完成一个任务。
- ☐ 不同任务确实使用不同配置，而不是只更改屏幕文字。
- ☐ 碰撞、燃料、失败条件和暂停行为可以被稳定复现。
- ☐ 能口头解释主要类的职责、update 循环、dt、碰撞公式与状态机。

2. 开发环境与首次运行

2.1 文件准备

1. 解压 OrbitalRescueStarter.zip，确认顶层直接包含 CMakeLists.txt、src、include 和 missions。
2. 在 CLion 中选择 Open，打开整个 OrbitalRescueStarter 文件夹，而不是只打开 main.cpp。
3. 等待 CMake 配置。第一次会通过 FetchContent 下载 raylib 5.5，因此需要网络。
4. 选择 orbital_rescue 运行目标并点击 Run。
5. 窗口出现后按 Enter 开始，使用 W、A、D、左右方向键飞行。

按键	作用	注意
Enter	开始任务	只在 Briefing 状态生效
W / Up	主发动机	沿飞船机头方向推力
A / D	侧向推进器	沿飞船局部右方向正负推力
Left / Right	旋转	改变角速度，而不是直接锁定角度
P	暂停	当前由 main.cpp 中的 bool 管理
R	重置	恢复初始位置、燃料、任务状态

2.2 运行基线验收

- ☐ 窗口大小约为 1280 x 760，能看到星空、飞船、空间站和遥测面板。
- ☐ 按 W 时飞船出现尾焰并逐渐加速，燃料下降。
- ☐ 按左右方向键后，飞船角度和角速度发生变化。
- ☐ 飞出边界或超过 180 秒会失败；按 R 可恢复。
- ☐ 满足距离、速度和角度条件时显示 DOCKING COMPLETE。

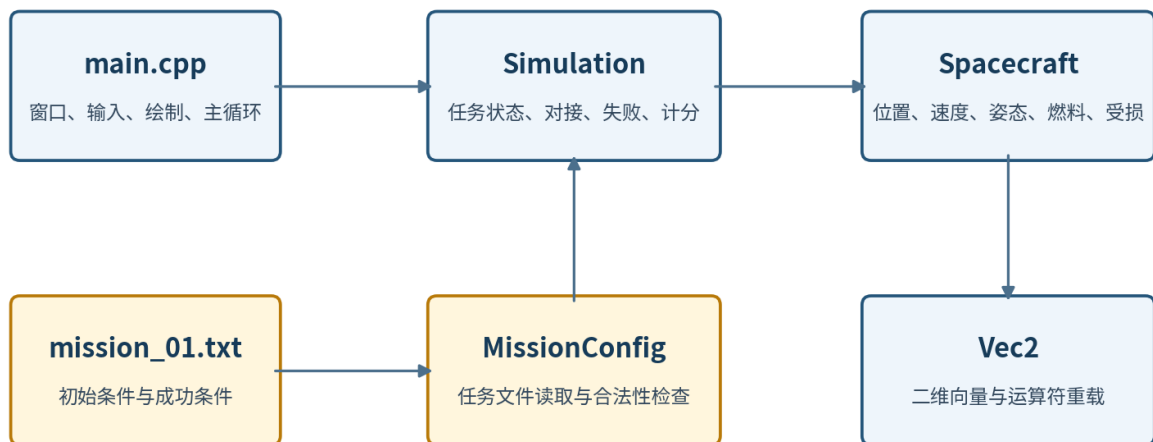
先不要立即改代码 先完整玩 3 次，记录“最难控制的量”以及“当前碰撞表现哪里不合理”。这会帮助你理解后面任务为什么存在。

2.3 常见配置问题

现象	优先检查	处理
CMake 找不到编译器	CLion Toolchains	选择 MinGW 或已安装的 GCC/Clang
下载 raylib 失败	网络、代理、Git	确认终端可运行 git；重新 Reload CMake Project

现象	优先检查	处理
找不到 raylib.h	CMake 尚未成功	不要手动复制头文件，先查看 CMake 错误
运行后立即退出	Run 配置或运行目录	确认目标是 orbital_rescue；查看控制台错误
任务文件找不到	Working directory	后续文件读取应基于项目运行目录并报告路径

3. 现有工程结构与程序流



设计目标：输入、模拟状态与绘制相互分离；GUI 不直接篡改物理状态。

文件	当前职责	后续主要修改
src/main.cpp	创建窗口、读取按键、调用 update、绘制全部 GUI	按钮、页面状态、轨迹绘制、任务选择、结果页
include/Spacecraft.hpp / src/Spacecraft.cpp	飞船状态、推进器、燃料、积分、受损	配置初始化、外力接口、碰撞速度修改、故障效果
include/Simulation.hpp / src/Simulation.cpp	任务状态、对接判定、失败、计分	DockingLimits、碰撞处理、引力、统计、自动驾驶接入
include/Vec2.hpp	二维向量基本运算	减法赋值、点积扩展、保护与测试
missions/mission_01.txt	任务故事与初始条件	变成可可靠读取的配置；增加 mission_02/03
CMakeLists.txt	C++20、下载 raylib、构建目标	加入新增 .cpp；可选测试目标

3.1 每帧发生什么

6. Raylib 读取键盘和鼠标事件。
7. readControls() 生成 ControlInput。
8. main() 计算 dt，并调用 Simulation::update(dt, input)。
9. Simulation 更新任务时间，再调用 Spacecraft::update。
10. Spacecraft 根据飞船朝向计算 forward/right，得到推力、加速度、速度和位置。
11. Simulation 检查对接与失败。
12. main.cpp 只读取状态并绘制飞船、空间站、遥测和页面信息。

架构原则 Simulation 和 Spacecraft 属于“模型”；main.cpp 的绘制函数属于“视图/输入层”。

绘制函数应读取状态，但不应该偷偷修改位置、燃料或任务结果。

3.2 当前故意保留的技术债

问题	为什么不合理	后续对应任务
进入捕获区后只调用 <code>craft_.update(0.05,{})</code>	这只是再推进一小步，并没有反射速度，不能形成真正反弹	任务 6
飞船留在捕获区会每帧扣血	缺少 <code>contact flag</code> 或 <code>collision cooldown</code>	任务 6
初始条件硬编码在 <code>Spacecraft::reset</code>	<code>mission_01.txt</code> 当前没有被程序读取	任务 1-2
对接限制散落为 <code>constexpr</code>	无法由任务配置改变，也容易重复	任务 3
暂停只由 <code>main</code> 的局部 <code>bool</code> 管理	页面状态变多后会难以维护	任务 9-10
速度安全使用绝对速度	空间站当前静止可用，但扩展移动目标时应使用相对速度	扩展项

4. 必要物理与数学模型

4.1 坐标与局部方向

世界坐标中 x 向右、y 向上；屏幕坐标的 y 向下，因此 `worldToScreen()` 使用 `screenY = centerY - worldY`。飞船角度 `angleRadians = 0` 时机头向世界坐标 +x。

现有程序的局部坐标方向

```
forward = { cos(angle), sin(angle) }
right   = {-sin(angle), cos(angle) }
force   = forward * mainThrust + right * lateralThrust
```

4.2 欧拉积分

```
acceleration = force / mass
velocity += acceleration * dt
position += velocity * dt
```

这是半隐式 Euler 积分：先更新速度，再用新速度更新位置。它简单且适合本项目，但 `dt` 过大会导致轨迹跳动、穿透碰撞区或控制器不稳定。现有代码将 `dt` 限制为 0.033 秒，约等于最差 30 FPS。

必须能解释 “限制 `dt`” 不是为了让电脑更快，而是为了避免窗口卡顿后下一帧一次跨越太远，使物理计算失真。

4.3 对接判定

条件	当前值	含义
距离	$\leq 38 \text{ m}$	进入捕获区
速度	$\leq 8 \text{ m/s}$	不能高速撞击
角度误差	$\leq 0.25 \text{ rad}$ ，约 14.3°	飞船机头需要基本指向 +x

注意：三个条件必须同时成立。`insideCaptureZone` 只是说明“接触到了对接区”，并不代表对接成功。

4.4 碰撞反射公式

设 n 为从碰撞表面指向飞船的单位法向量， v 为碰撞前速度， e 为恢复系数（0 表示完全无弹性，1 表示完全弹性）。只在 $v \cdot n < 0$ ，即飞船正在朝表面运动时反射：

```
normalSpeed = dot(v, n)
if (normalSpeed < 0) {
    vAfter = v - n * ((1 + restitution) * normalSpeed)
```

```
}
```

不能把整个速度简单取反，否则切向速度也会被错误翻转。还必须把飞船推出表面或设置接触冷却，否则下一帧仍在重叠区内。

4.5 月球引力

```
r = moonPosition - craftPosition  
a = mu * r / |r|^3
```

实现时先计算 `distanceSquared`，再设置最小半径保护。不要先 `normalized()` 后又除以 `distanceSquared` 而忘记零距离情况。时间倍率增大时，数值误差也会放大。

5. 总体开发路线



每个阶段都必须：提交可运行版本 + 验收记录 + 一次有意义的 Git commit

阶段	建议用时	结束时必须能展示
0 基线与版本控制	1 次	原始项目运行；第一次 commit；记录已知问题
1 C++ 核心	2 次	配置文件驱动任务；轨迹尾迹；无硬编码限制
2 模拟工程	2 次	可复现碰撞反弹；月球引力；时间倍率
3 GUI	2 次	鼠标按钮、任务选择、结果页面
4 自动驾驶	2-3 次	至少一关可稳定自动对接
5 故障与测试	1-2 次	故障确实影响控制；完整测试与说明

范围控制 必须先完成阶段 1-3，再做自动驾驶。不要一开始就写自动驾驶，否则你会同时调试物理、GUI、任务配置和控制算法，问题无法定位。

6. 阶段 0：基线、Git 与阅读代码

任务 0A - 建立基线

13. 运行原始项目，不改任何代码。
14. 完成一次成功对接和一次失败撞击；记录时间、速度和燃料。
15. 创建 Git 仓库并提交：starter runs。
16. 在 NOTES.md 中写下至少 5 个你观察到的程序行为。

建议命令

```
git init
git add .
git commit -m "starter runs"
```

任务 0B - 代码阅读追踪

用 CLion 的 Find Usages 或 Ctrl+B 跟踪以下路径，并写一句话解释每个调用：

- main() -> readControls() -> ControllInput
- main() -> Simulation::update()
- Simulation::update() -> Spacecraft::update()
- Simulation::checkDocking() -> evaluateDocking()
- main() -> drawTelemetry() -> const Spacecraft&

自测问题	合格回答应包含
为什么 craft() 有 const 与非 const 两个版本？	只读绘制可以获得 const 引用；需要修改时才获得非 const 引用
ControllInput 为什么是 struct？	把同一帧的三类控制量作为一个清楚的数据对象传递
Vec2 为什么重载 +、-、*？	让物理公式接近数学表达，减少 x/y 重复代码
namespace {} 有什么作用？	让常量和辅助函数只在当前 .cpp 内可见

7. 阶段 1：任务配置与结构重构

任务 1 - 创建 MissionConfig

目标：让任务初始条件和成功条件来自文件，而不是散落在代码里。

建议的数据结构（可自行调整）

```
struct DockingLimits {
    double captureDistance{};
    double safeSpeed{};
    double safeAngleRadians{};
};

struct MissionConfig {
    std::string name;
    std::string callsign;
    Vec2 startPosition;
    Vec2 startVelocity;
    double startAngleRadians{};
    double startFuel{};
    double timeLimit{};
    DockingLimits docking;
};
```

建议新增文件：include/MissionConfig.hpp 与 src/MissionConfig.cpp。加载函数不要依赖 GUI：

```
MissionConfig loadMissionConfig(const std::string& filename);
```

实现要求

- 使用 `std::ifstream` 打开文件；打不开时抛出 `std::runtime_error`，并包含文件名。
- 逐行读取；跳过空行和以 `#` 开头的注释。
- 将 `key = value` 拆分为键和值，并清除首尾空格。
- 数值转换失败时报告具体 key 和原文本。
- 检查燃料、时间限制、捕获距离和安全速度必须大于 0。
- 将 `alignment_error_degrees` 转换成弧度后存储。
- 未知键可以选择忽略并警告，或直接报错；必须保持一致。

文件格式提醒 原 `mission_01.txt` 同时包含故事文字、标题和类似 “distance <= 38 m” 的自然语言。最稳妥的做法是把可读取参数统一改成 `key = value`，故事改用 `#` 注释或单独的 `description` 字段。

推荐的机器可读格式

```
name = ORBITAL RESCUE 01
callsign = KESTREL
position_x = -260
position_y = 80
```

```
velocity_x = 10
velocity_y = -2
start_angle_deg = 8.6
fuel = 600
time_limit = 180
capture_distance = 38
safe_speed = 8
safe_angle_deg = 14
```

验收测试

测试	操作	预期
正常文件	读取 mission_01.txt	所有字段正确进入 MissionConfig
文件不存在	改成错误路径	程序显示明确错误，而不是静默使用默认值
缺少 fuel	删除 fuel 行	加载失败并说明缺少 fuel
fuel = abc	提供非数值	指出 fuel 转换失败
safe_speed = -1	提供非法范围	拒绝启动任务

任务 2 - 用配置重置飞船

Spacecraft::reset() 目前写死位置、速度和燃料。将其改成接收配置或专门的初始状态。不要让 Spacecraft 自己打开任务文件；文件读取属于配置层，Spacecraft 只接收已经验证过的数据。

```
void reset(const MissionConfig& config);
// 或: void reset(const SpacecraftInitialState& state);
```

验收：修改 mission_01.txt 的 position_y 或 fuel 后重新运行，屏幕行为必须真正改变。

任务 3 - 集中 DockingLimits

删除 Simulation.cpp 中 kCaptureDistance、kSafeDockingSpeed 和 kSafeAngle 的重复常量。Simulation 保存一份当前任务的 DockingLimits，并在 evaluateDocking()、GUI 提示和任务结果中复用。

不要做的事 不要让 main.cpp 仍然写死 “8 m/s、14 degrees”。否则模拟判定和屏幕提示可能不一致。

8. 阶段 1：飞行轨迹与 std::vector

任务 4 - 保存最近 300 个位置

在 Simulation 中新增 std::vector<Vec2> trail_。只有 Running 状态且飞船实际更新时才加入位置。重置任务时清空。

```
std::vector<Vec2> trail_;
static constexpr std::size_t kMaxTrailPoints = 300;
```

基本算法：push_back 当前位置；当 size() 超过上限时删除最旧位置。对于 300 个点，erase(begin()) 足够；进阶可使用 std::deque 或环形缓冲区。

任务 5 - 绘制渐隐轨迹

main.cpp 读取 const std::vector<Vec2>&，将相邻点转换为屏幕坐标后绘制线段。越新的线段 alpha 越大。

提示，不是完整答案

```
alpha = static_cast<float>(i) / (trail.size() - 1)
DrawLineV(worldToScreen(trail[i-1]),
           worldToScreen(trail[i]),
           Fade(SKYBLUE, alpha));
```

边界情况	正确处理
trail.size() < 2	不绘制，避免 size()-1 下溢或除零
暂停	不新增点，但仍显示已有轨迹
重启	轨迹清空
时间倍率 5x	不要每个物理子步无限加点；可按距离或时间采样
任务结束	轨迹保留到结果页，方便复盘

阶段 1 Git 提交建议：mission config loader、flight trail。

9. 阶段 2：真实碰撞响应

任务 6 - 设计碰撞几何

先明确空间站的“可碰撞区域”。入门实现可把捕获区看作以 `dockingPortPosition` 为圆心、半径 `captureDistance` 的圆。飞船中心进入圆内即发生接触。

- 17. 计算 `offset = craft.position - port.position`。
- 18. 计算 `distance = offset.magnitude()`。
- 19. 若 `distance < radius`，则 `normal = offset.normalized()`。
- 20. 若 `distance` 极小，使用一个固定法向量，避免 `normalized()` 返回零向量。
- 21. 计算 `penetration = radius - distance`，把飞船沿 `normal` 推出表面。
- 22. 只在 `dot(velocity, normal) < 0` 时反射速度。
- 23. 根据撞击法向速度计算损伤，而不是直接使用总速度。

接口问题 `Spacecraft` 当前没有 `setPosition/setVelocity`。不要为了方便把所有成员改成 `public`。应提供受控接口，例如 `resolveCollision(normal, penetration, restitution)` 或 `applyImpulse(...)`。

避免连续扣血

至少选择一种方法：

- **接触标志**：上一帧不在接触区、本帧进入时才算新碰撞。
- **冷却时间**：碰撞后 0.3-0.5 秒内不重复扣血。
- **位置修正**：每次都把飞船推出圆外，并且只在向内速度时处理。

最佳做法通常是“位置修正 + 向内速度检查”，并保留 `contact flag` 用于统计撞击次数。

碰撞验收

场景	预期结果
慢速且对准进入	任务成功，不反弹
高速正面撞击	明显反弹、扣一次血、记录 <code>impact speed</code>
切向擦过	主要保留切向速度，不应整向量反转
停在边界附近	船体不会每帧持续下降
超高速撞击	船体降至 0 后进入 <code>Failed</code>

10. 阶段 2：月球引力与时间倍率

任务 7 - 增加外力接口

不要把“月球位置”直接硬编码在 `Spacecraft::update()`。更清楚的设计是由 `Simulation` 计算环境加速度，再传给 `Spacecraft`。

```
void update(double dt,
            const ControlInput& input,
            const Vec2& externalAcceleration);
```

这样以后可以添加多个天体、空间站移动、故障推力或自动驾驶，而不会让 `Spacecraft` 知道整个世界。

引力实现要求

- 在 `Simulation` 中保存 `moonPosition` 和 `mu`。
- 使用 `r = moonPosition - craft.position()`。
- 对 `distanceSquared` 设置最小值，例如不小于 `minRadius2`。
- 先用很小的 `mu` 调试，确认方向正确，再逐步提高。
- 画出月球和影响提示；否则玩家无法理解飞船为何偏转。

测试技巧 暂时关闭所有推进器，把初速度设为 0。飞船应缓慢朝月球方向加速。若远离月球，`r` 的方向写反了。

任务 8 - 时间倍率 1x / 2x / 5x

最简单的 `dt * multiplier` 在 5x 时可能变得不稳定。更可靠的方法是固定小步长子步进：

```
simulatedDt = realDt * timeScale
while (simulatedDt > 0) {
    step = min(simulatedDt, maxPhysicsStep)
    updatePhysics(step)
    simulatedDt -= step
}
```

要求	验收
暂停时倍率不推进任务时间	按 P 后 <code>missionTime</code> 不变
5x 下不穿过对接区	高速接近仍能触发碰撞或对接
GUI 显示当前倍率	玩家无需猜测
切换倍率不修改速度本身	只改变模拟时间推进速度

11. 阶段 3：可点击 GUI 与程序页面

任务 9 - 通用 Button

使用 raylib 的 `Rectangle`、`GetMousePosition()`、`CheckCollisionPointRec()` 和 `IsMouseButtonPressed()`。建议写一个轻量 `Button` 结构或 `drawButton` 函数。

```
struct Button {
    Rectangle bounds;
    std::string label;
    bool enabled{true};

    bool clicked() const;
    void draw() const;
};
```

按钮至少需要三种视觉状态：普通、悬停、禁用。点击事件只在按下瞬间触发，不要使用 `IsMouseButtonDown` 导致每帧重复执行。

任务 10 - 应用页面状态

`MissionState` 表示任务内部状态，但应用还需要任务选择和结果页。可以新增 `AppScreen`：

```
enum class AppScreen {
    MainMenu,
    MissionSelect,
    Briefing,
    Flight,
    Results
};
```

避免状态爆炸 `AppScreen` 负责“显示哪个页面”；`MissionState` 负责“当前任务发生了什么”。不要把暂停、自动驾驶、菜单、碰撞全部塞进一个巨大 `enum`。

任务 11 - 三个任务与结果页

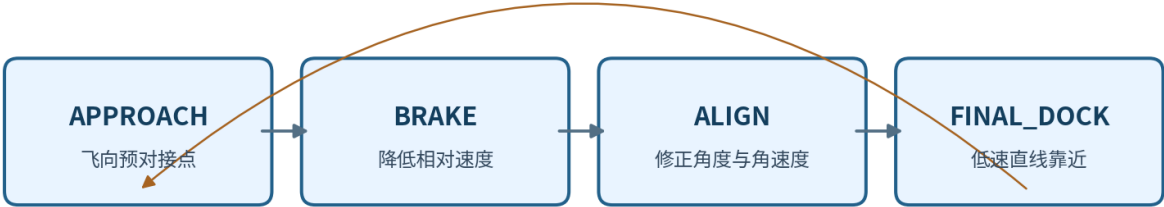
任务建议	特点	训练点
Mission 01 - Standard Rescue	原始条件，无引力	基础操纵与配置
Mission 02 - Lunar Drift	较少燃料，明显月球引力	外力补偿与时间倍率
Mission 03 - Damaged Kestrel	某个推进器受限或有燃料泄漏	故障与自动驾驶鲁棒性

结果页至少统计：`startFuel - finalFuel`、`missionTime`、`maxSpeed`、`impactSpeed`、`hullIntegrity`、`score`。统计量应在 `Simulation` 中维护，结果页只负责显示。

按钮	何时可用	行为
Start	Briefing	进入 Flight 并开始计时
Pause/Resume	Flight	冻结物理和任务时间
Restart	Flight/Results	按当前任务重新初始化
Autopilot	Flight	切换自动驾驶；显示 ON/OFF
Back	Briefing/Results	回任务选择，不直接退出程序

12. 阶段 4：自动对接系统

自动驾驶只能输出 **ControllInput**，禁止直接设置 **position** 或 **velocity**



状态切换依赖距离、速度、角度和稳定时间；失败时允许回退，而不是只向前跳转。

自动驾驶是本工程最能检验综合能力的部分。它必须读取飞船和任务状态，输出同样的 **ControllInput**。人类控制与自动控制应当可以替换：

```
ControlInput input = autopilotEnabled
    ? autopilot.update(simulation, dt)
    : readManualControls();
```

12.1 建议类接口

```
enum class AutopilotMode {
    Approach,
    Brake,
    Align,
    FinalDock,
    Complete,
    Abort
};

class Autopilot {
public:
    void reset();
    ControlInput update(const Simulation& sim, double dt);
    AutopilotMode mode() const;
};
```

12.2 每个状态的任务

状态	目标	典型切换条件
APPROACH	飞向端口前方约 80-120 m 的预对接点	距预对接点较近，或需要开始制动
BRAKE	将相对速度降低到可控范围	速度低于阈值且位置基本正确

状态	目标	典型切换条件
ALIGN	角度接近 0，角速度接近 0	角度和角速度持续稳定一段时间
FINAL_DOCK	沿对接轴低速接近	成功、碰撞或偏离过大
ABORT	远离端口并稳定飞船	恢复安全后回 APPROACH 或由玩家接管

12.3 角度控制

首先计算目标方向 $\text{desiredAngle} = \text{atan2}(\text{target.y} - \text{p.y}, \text{target.x} - \text{p.x})$ 。角度误差必须 wrap 到 $[-\pi, \pi]$ ，否则从 $+179^\circ$ 到 -179° 时会错误地旋转 358° 。

```
angleError = wrapAngle(desiredAngle - currentAngle)
rotationCommand = kp * angleError - kd * angularVelocity
rotationInput = clamp(rotationCommand, -1.0, 1.0)
```

先只调角度控制：关闭平移推进器，让飞船从多个初始角度稳定指向目标。若不断来回摆动，kp 过大或 kd 太小；若反应非常慢，kp 太小。

12.4 平移控制

一种可行思路是先计算世界坐标中的 $\text{desiredAcceleration}$ ，再投影到飞船局部坐标：

```
forwardCommand = dot(desiredAcceleration, forward)
lateralCommand = dot(desiredAcceleration, right)
```

但主发动机只能产生正向推力，不能直接倒车。需要转向制动，或限制主发动机命令并使用侧推辅助。最终对接阶段应设置速度曲线：距离越近，允许速度越低。

禁止作弊 不得在自动驾驶中直接调用 `setPosition`、`setVelocity` 或把飞船吸到端口。自动驾驶的价值就在于它必须受相同推力、燃料和转动惯性限制。

13. 自动驾驶调试顺序与验收

13.1 分层调试

- 24. 只做“指向固定点”：位置不动，验证角度控制。
- 25. 只做“降低速度”：设置简单初速度，验证制动逻辑。
- 26. 飞向预对接点，但暂时不进入捕获区。
- 27. 加入 ALIGN，并要求连续稳定 0.5 秒后才能切换。
- 28. 最后开启 FINAL_DOCK，速度限制先设得很保守。
- 29. 通过后再加入月球引力、低燃料和故障。

13.2 必须记录的诊断量

- 当前 AutopilotMode；
- 距目标点的距离；
- 目标角度、当前角度、angleError；
- 当前速度及沿对接轴/垂直轴的速度分量；
- rotationInput、mainThrottle、lateralThrottle；
- 状态切换原因。

可以先输出到控制台，稳定后再放入 GUI 调试面板。不要只凭“看起来差不多”调参数。

13.3 自动驾驶验收矩阵

初始情况	最低要求
原始 Mission 01	连续 5 次至少成功 4 次
初始角度 +90°	先稳定转向，再接近；不能直接冲撞
速度比原始高 50%	能够进入 BRAKE，或安全 ABORT
燃料只剩 50%	不一定成功，但必须合理失败并报告原因
月球引力开启	能补偿偏移，至少成功 3/5 次
玩家中途关闭自动驾驶	立即恢复手动输入，不改变当前状态

14. 阶段 5：故障系统

故障必须改变物理或信息，而不是仅显示一条文字。建议在任务开始 15 秒后，从允许的故障中随机选择一个，并记录触发时间。

故障	真实效果	实现提示
主发动机降至 55%	最大主推力乘 0.55	FaultState 影响 thrust multiplier
左侧推失效	一个方向 lateral command 无效	根据命令正负屏蔽一侧
燃料泄漏	即使不点火也持续耗油	每秒减少 leakRate * dt
陀螺仪噪声	显示或自动驾驶读取的角速度带噪声	区分 true state 与 sensor reading
遥测延迟	GUI 显示过去的状态	保存时间序列缓冲，不要延迟真实物理
转动阻尼失效	angularVelocity 不再正常衰减	调整 damping factor

设计建议 用 enum class FaultType 和一个 FaultState 管理故障。不要在 Spacecraft::update() 中堆很多随机 if；随机选择只做一次，效果每帧稳定应用。

```
enum class FaultType {
    None,
    MainEngineLimited,
    LeftThrusterFailed,
    FuelLeak,
    GyroNoise,
    TelemetryDelay,
    RotationDampingFailure
};
```

验收：同一个随机种子或调试按钮应能强制触发指定故障，便于复现。不能只能“等随机”。

15. 测试策略

15.1 纯逻辑测试

优先测试不依赖 raylib 窗口的部分。可以使用 assert，或新增一个简单 test 可执行目标。

```
void testVec2Magnitude() {
    Vec2 v{3.0, 4.0};
    assert(std::abs(v.magnitude() - 5.0) < 1e-9);
}
```

模块	至少测试
Vec2	magnitude、normalized、dot、零向量
MissionConfig	正常、缺失、非法数值、非法范围
wrapAngle	0、pi 附近、跨越 -pi/pi
DockingResult	边界内外、速度等于限制、角度等于限制
Collision	正面、切向、远离表面、零距离
Score	时间、燃料、船体对分数的影响

15.2 场景测试

编号	步骤	预期	结果
S01	启动后不按键 10 秒	位置按初速度变化，燃料不下降	
S02	持续 W 3 秒	速度和燃料有连续变化	
S03	高速撞击端口	反弹且只扣一次血	
S04	暂停 5 秒	任务时间、位置、燃料均不变	
S05	切换 5x	显示倍率，物理无明显穿透	
S06	修改任务文件 fuel	新任务燃料改变	
S07	自动驾驶 Mission 01	完成安全对接	
S08	强制主发动机故障	推力明显下降并显示故障	

15.3 调试纪律

- 一次只修改一个可验证行为；
- 先打印状态和中间变量，再猜原因；

- 每次修复后重新运行导致问题的同一场景；
- 不要用“再加一个 if”掩盖状态设计问题；
- 编译 warning 也要处理，尤其是符号/无符号比较和窄化转换。

16. 常见故障定位表

现象	优先怀疑	检查方法
飞船朝错误方向加速	角度符号、forward/right 定义	angle=0、pi/2 时手算向量
自动驾驶绕圈	角度误差没 wrap，或 kd 太小	显示 angleError 和 angularVelocity
接近端口突然穿过去	dt 太大、无子步进	输出每帧位置差和 timeScale
碰撞连续扣血	仍在重叠区、无向内速度检查	显示 penetration 和 dot(v,n)
尾迹越跑越慢	vector 无限增长	检查最大点数与清理逻辑
任务配置改了没效果	仍使用硬编码 reset	搜索 -260、600、38、8、0.25
暂停后仍耗油	输入或 update 仍执行	检查 pause 分支和 missionTime
按钮一次点击触发多次	使用 IsMouseButtonDown	改用 IsMouseButtonPressed
5x 时自动驾驶失控	控制器与物理步长过大	固定小步长；控制输出限幅
故障无法复现	随机选择没有 seed/强制模式	增加 debug fault selector

搜索硬编码 阶段 1 完成后，在整个项目搜索 260、600、38、8、0.25、180。除默认配置或测试外，不应再有多处相同任务参数。

17. 最终提交物与工程质量

17.1 必交文件

- ☐ 完整源码：src、include、missions、CMakeLists.txt。
- ☐ README.md：构建、控制、任务、已知限制。
- ☐ TEST_REPORT.md：测试矩阵、结果、失败案例。
- ☐ DESIGN.md：类职责、状态机、关键公式、设计取舍。
- ☐ 至少三份任务配置。
- ☐ 至少一段自动驾驶完整对接录屏。

17.2 README 最低内容

- 依赖与构建步骤；
- 键盘和鼠标控制；
- 三种任务的区别；
- 自动驾驶算法概述；
- 故障系统；
- 仍未解决的问题；
- 你本人完成了哪些模块。

17.3 代码质量检查

检查项	合格标准
命名	类 PascalCase；函数/变量一致且有含义
const	只读函数和引用尽量 const
头文件	使用 #pragma once；不在头文件放大段实现
所有权	无不必要 new/delete；优先值语义和标准容器
魔法数字	任务参数集中配置；物理常量有命名
函数长度	主循环和 update 不成为数百行巨大函数
依赖方向	模型不 include raylib.h；绘制层可依赖模型
错误处理	文件错误有清楚信息，不静默失败

18. 评分标准（100 分）

领域	分值	评分重点
编译与正确性	15	干净构建、稳定运行、无明显越界/除零/无限循环
C++ 结构	15	类职责、头源分离、const、接口设计、无公共状态滥用
容器与文件 I/O	12	MissionConfig、合法性检查、轨迹容器、错误报告
物理与数值	15	碰撞法向、一次扣血、引力保护、时间子步进
GUI 与状态	13	鼠标按钮、多页面、暂停/重开一致、结果页
自动驾驶	20	FSM、角度控制、平移控制、稳定切换、无作弊
故障、测试与说明	10	真实故障、可复现测试、设计说明和答辩

总分	能力判断
< 50	基础语法能写，但多文件工程、状态与调试仍不稳定。
50-64	可完成普通入门题；需要加强结构设计与验证。
65-79	暑假预习合格，能够完成中等规模第一年项目。
80-89	工程能力较强，能独立组织 GUI 模拟和控制逻辑。
90-100	不仅功能完整，而且设计、测试和解释均成熟。

评分重点 不是功能越多分越高。一个稳定、结构清楚、可解释的自动驾驶，比十个未经测试的花哨功能更有价值。

19. 建议开发安排（10 次）

次数	主要任务	本次结束条件	建议 commit
1	运行、玩项目、读调用链、Git	能解释每帧程序流	starter runs
2	MissionConfig 与验证	任务文件真正改变初始状态	mission config loader
3	DockingLimits、轨迹 vector	参数无重复；渐隐尾迹	flight trail
4	碰撞几何与位置修正	高速撞击只扣一次并反弹	collision response
5	月球引力、时间子步进	1x/2x/5x 稳定	gravity and time warp
6	Button 与 AppScreen	鼠标可开始、暂停、重开	gui buttons
7	三任务与结果页	可选关；统计正确	mission results
8	Autopilot 角度/制动	能指向并减速	autopilot approach
9	状态机与最终对接	Mission 01 成功率 >= 80%	autopilot docking
10	故障、测试、文档	完整提交与自评	cleanup and report

每次结束前用 10 分钟回答：本次新增了什么状态？谁拥有这个状态？谁可以修改它？如何证明它正确？

20. 项目答辩问题

完成项目后，不看代码尝试回答。无法解释的部分通常也是下一步需要补强的 C++ 能力。

1. 为什么 `Vec2::normalized()` 对接近零的向量需要保护？

答：

2. 为什么总质量随燃料消耗变化会影响加速度？当前模型是否真的体现了这一点？

答：

3. 为什么渲染函数应接受 `const Spacecraft&`？

答：

4. 为什么任务文件读取不应放在 `Spacecraft` 类内部？

答：

5. 为什么 `vector erase(begin())` 是 $O(n)$ ？本项目为什么仍可接受？

答：

6. 碰撞反射为什么只处理法向速度分量？

答：

7. 恢复系数 e 对碰撞结果有什么影响？

答：

8. 为什么 5x 时间倍率最好使用子步进？

答：

9. `AppScreen` 与 `MissionState` 有什么区别？

答：

10. 为什么按下按钮应该用 `IsMouseButtonPressed` 而不是 `Down`？

答：

11. 自动驾驶为什么需要 `wrapAngle`？

答：

12. 比例控制器 k_p 过大和阻尼 k_d 过小分别会出现什么现象？

答：

13. 自动驾驶失败时为什么需要 Abort 状态？

答：

14. 如何证明自动驾驶没有直接修改位置或速度？

答：

15. 如果空间站也在移动，对接速度应该怎样计算？

答：

21. 进阶扩展（完成主线后再选）

扩展	难度	价值
移动空间站与相对速度	中	更接近 rendezvous；改进对接判定
相机跟随与缩放	中	练习坐标变换与 GUI
粒子推进器尾焰	中	练习对象池和生命周期
传感器噪声与滤波	中高	连接 Python/信号处理思维
PID 自动驾驶	高	在 PD 基础上处理稳态偏差
轨道力学近似	高	用中心引力替代弱阻尼平面模型
任务回放文件	高	记录输入与状态，支持 deterministic replay
单元测试框架 Catch2	中	建立正式测试工程

主线优先 进阶功能不得以破坏主线稳定性为代价。每加一个扩展，都应增加对应测试和开关。

22. 最终检查清单

构建与运行

- ☐ 新目录能 CMake 配置
- ☐ 无编译错误
- ☐ 主要 warning 已处理
- ☐ 运行目录能找到 missions

任务与模拟

- ☐ 三任务配置可读
- ☐ 对接限制来自配置
- ☐ 碰撞只处理一次
- ☐ 引力有零距离保护
- ☐ 时间倍率稳定

GUI

- ☐ 所有按钮可点击
- ☐ 暂停冻结全部模拟
- ☐ 任务选择有效
- ☐ 结果统计正确
- ☐ 错误信息可见

自动驾驶

- ☐ 使用 ControllInput
- ☐ 有明确 FSM
- ☐ 角度误差 wrap
- ☐ 控制输出限幅
- ☐ 至少 4/5 成功率

工程质量

- ☐ 模型不依赖 raylib
- ☐ 没有公共成员滥用
- ☐ 无大量魔法数字
- ☐ README/TEST_REPORT/DESIGN 完整
- ☐ Git 历史有阶段节点

23. 完成记录

项目	记录
开始日期	_____
完成日期	_____
最终 Git commit	_____
手动对接最佳成绩	_____
自动对接成功率	_____
自评分数 / 100	_____
最困难的模块	_____
最满意的改进	_____

最终反思（建议 150-250 字）：

项目完成定义 当你能够现场修改一个任务参数、重新运行、解释程序如何把配置传到飞船和判定系统，并展示一次手动及一次自动对接时，这个工程才真正完成。

MISSION READY

Build. Test. Dock. Explain.

对应工程：OrbitalRescueStarter

指导书版本 1.0